

6)。以下 s 行每行为一个挂坠的重量 w_i ($1 \leq w_i \leq 1000$)。输入保证不存在天平的宽度恰好在 $r-10^{-5}$ 和 $r+10^{-5}$ 之间（这样可以保证不会出现精度问题）。对于每组数据，输出最优天平的宽度。如果无解，输出-1。你的输出和标准答案的绝对误差不应超过 10^{-8} 。

【分析】

如果把挂坠和木棍都作为结点，则一个天平对应一棵二叉树，如题目中给出的，挂坠为 1, 1, 2 的 3 个天平如图 7-12 所示。

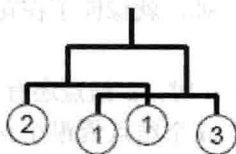


图 7-11 子天平相互重叠

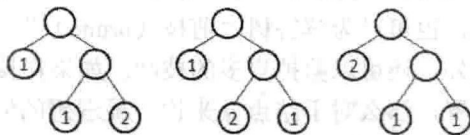


图 7-12 与天平对应的二叉树

对于一棵确定二叉树，可以计算出每个挂坠的确切位置，进而计算出整个天平的宽度，所以本题的核心任务是：枚举二叉树。

如何枚举二叉树呢？最直观的方法是沿用回溯法框架，每次选择两个结点组成一棵子树，递归 $s-1$ 层即可。以 4 个挂坠 1, 1, 2, 3 为例，下面是解答树的一部分（每个结点的子树并没有全部画出），如图 7-13 所示。

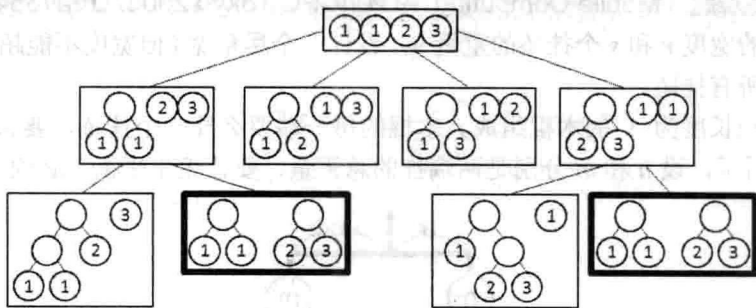


图 7-13 解答树

上面的方法已经足够解决本题，但还有优化的余地，因为有些二叉树被枚举了多次（如图 7-13 中的两个粗框结点）。

推荐的枚举方法是：自顶向下构造，每次枚举左子树用到哪个子集，则右子树就是使用剩下的子集。在第 9 章中会专门讨论“枚举子集”的高效算法，建议读者在学习之后重新实现本题。

7.5 路径寻找问题

在第 6 章中曾经介绍过图的遍历。很多问题都可以归结为图的遍历，但这些问题中的图却不是事先给定、从程序读入的，而是由程序动态生成的，称为隐式图。本节和前面介

绍的回溯法不同：回溯法一般是要找到一个（或者所有）满足约束的解（或者某种意义上最优解），而状态空间搜索一般是要找到一个从初始状态到终止状态的路径。

提示 7-15：路径寻找问题可以归结为隐式图的遍历，它的任务是找到一条从初始状态到终止状态的最优路径，而不是像回溯法那样找到一个符合某些要求的解。

八数码问题。编号为 1~8 的 8 个正方形滑块被摆成 3 行 3 列（有一个格子留空），如图 7-14 所示。每次可以把与空格相邻的滑块（有公共边才算相邻）移到空格中，而它原来的位置就成为了新的空格。给定初始局面和目标局面（用 0 表示空格），你的任务是计算出最少的移动步数。如果无法到达目标局面，则输出 -1。

2	6	4
1	3	7
	5	8

8	1	5
7	3	6
4		2

图 7-14 八数码问题举例

样例输入：

```
2 6 4 1 3 7 0 5 8
8 1 5 7 3 6 4 0 2
```

样例输出：

31

【分析】

不难把八数码问题归结为图上的最短路问题，图的“结点”就是 9 个格子中的滑块编号（从上到下、从左到右把它们放到一个包含 9 个元素的数组中）。根据第 6 章的讲解，无权图上的最短路问题可以用 BFS 求解，代码如下：

```
typedef int State[9];           //定义“状态”类型
const int maxstate = 1000000;
State st[maxstate], goal;      //状态数组。所有状态都保存在这里
int dist[maxstate];           //距离数组
//如果需要打印方案，可以在这里加一个“父亲编号”数组 int fa[maxstate]

const int dx[] = {-1, 1, 0, 0};
const int dy[] = {0, 0, -1, 1};
//BFS，返回目标状态在 st 数组下标
int bfs() {
    init_lookup_table();       //初始化查找表
    int front = 1, rear = 2;   //不使用下标 0，因为 0 被看作“不存在”
    while(front < rear) {
        State& s = st[front]; //用“引用”简化代码
```

```

if(memcmp(goal, s, sizeof(s)) == 0) return front; //找到目标状态，成功返回
int z;
for(z = 0; z < 9; z++) if(!s[z]) break; //找“0”的位置
int x = z/3, y = z%3; //获取行列编号（0~2）
for(int d = 0; d < 4; d++) {
    int newx = x + dx[d];
    int newy = y + dy[d];
    int newz = newx * 3 + newy;
    if(newx >= 0 && newx < 3 && newy >= 0 && newy < 3){ //如果移动合法
        State& t = st[rear];
        memcpy(&t, &s, sizeof(s)); //扩展新结点
        t[newz] = s[z];
        t[z] = s[newz];
        dist[rear] = dist[front] + 1; //更新新结点的距离值
        if(try_to_insert(rear)) rear++; //如果成功插入查找表，修改队尾指针
    }
}
front++; //扩展完毕后再修改队首指针
}
return 0; //失败
}

```

注意，此处用到了 `cstring` 中的 `memcmp` 和 `memcpy` 完成整块内存的比较和复制，比用循环比较和循环赋值要快。主程序很容易实现：

```

int main(){
    for(int i = 0; i < 9; i++) scanf("%d", &st[1][i]); //起始状态
    for(int i = 0; i < 9; i++) scanf("%d", &goal[i]); //目标状态
    int ans = bfs(); //返回目标状态的下标
    if(ans > 0) printf("%d\n", dist[ans]);
    else printf("-1\n");
    return 0;
}

```

注意，应在调用 `bfs` 函数之前设置好 `st[1]` 和 `goal`。上面的代码几乎是完整的，唯一没有涉及的是 `init_lookup_table()` 和 `try_to_insert(rear)` 的实现。为什么会有这两项呢？还记得 BFS 中的“判重”操作吗？在 DFS 中可以检查 `idx` 来判断结点是否已经访问过；在求最短路的 BFS 中用 `d` 值是否为 -1 来判断结点是否访问过，不管用哪种方法，作用是相同的：避免同一个结点访问多次。树的 BFS 不需要判重，因为根本不会重复；但对于图来说，如果不判重，时间和空间都将产生极大的浪费。

如何判重呢？难道要声明一个 9 维数组 `vis`，然后执行 `if(vis[s[0]][s[1]][s[2]]...s[8])`？无论程序好不好看，9 维数组的每维都要包含 9 个元素，一共有 $9^9=387420489$ 项，太多了，数组开不下。实际的结点数并没有这么多（0~8 的排列总共只有 $9!=362880$ 个），为什么 9

维数组开不下呢？原因在于，这样的用法存在大量的浪费——数组中有很多项都没有被用到，但却占据了空间。

下面通过讨论 3 种常见的方法来解决这个问题，同时将它们用到八数码问题中。

第 1 种方法是：把排列“变成”整数，然后只开一个一维数组。也就是说，设计一套排列的编码（encoding）和解码（decoding）函数，把 0~8 的全排列和 0~362879 的整数一一对应起来。第 10 章中将详细讨论编码和解码问题，这里先给出代码以便读者形成一个感性认识：

```
int vis[362880], fact[9];
void init_lookup_table(){
    fact[0] = 1;
    for(int i = 1; i < 9; i++) fact[i] = fact[i-1] * i;
}
int try_to_insert(int s){
    int code = 0; //把 st[s]映射到整数 code
    for(int i = 0; i < 9; i++){
        int cnt = 0;
        for(int j = i+1; j < 9; j++) if(st[s][j] < st[s][i]) cnt++;
        code += fact[8-i] * cnt;
    }
    if(vis[code]) return 0;
    return vis[code] = 1;
}
```

尽管原理巧妙，时间效率也非常高，但编码解码法的适用范围并不大：如果隐式图的总结点数非常大，编码也将会很大，数组还是开不下。

第 2 种方法是使用哈希（hash）技术。简单地说，就是要把结点“变成”整数，但不必是一一对应。换句话说，只需要设计一个所谓的哈希函数 $h(x)$ ，然后将任意结点 x 映射到某个给定范围 $[0, M-1]$ 的整数即可，其中 M 是程序员根据可用内存大小自选的。在理想情况下，只需开一个大小为 M 的数组就能完成判重，但此时往往会有不同结点的哈希值相同，因此需要把哈希值相同的状态组织成链表，细节参见下面的代码：

```
const int hashsize = 1000003;
int head[hashsize], next[maxstate];
void init_lookup_table() { memset(head, 0, sizeof(head)); }
int hash(State& s){
    int v = 0;
    for(int i = 0; i < 9; i++) v = v * 10 + s[i]; //把 9 个数字组合成 9 位数
    return v % hashsize; //确保 hash 函数值是不超过 hash 表的大小的非负整数
}
int try_to_insert(int s){
    int h = hash(st[s]);
```

```

int u = head[h]; //从表头开始查找链表
while(u) {
    if(memcmp(st[u], st[s], sizeof(st[s]))==0) return 0; //找到了，插入失败
    u = next[u]; //顺着链表继续找
}
next[s] = head[h]; //插入到链表中
head[h] = s;
return 1;
}

```

哈希表的执行效率高，适用范围也很广。除了 BFS 中的结点判重外，还可以用到其他需要快速查找的地方。不过需要注意的是：在哈希表中，对效率起到关键作用的是哈希函数。如果哈希函数选取得当，几乎不会有结点的哈希值相同，且此时链表查找的速度也较快；但如果冲突严重，整个哈希表会退化成少数几条长长的链表，查找速度将非常缓慢。有趣的是，前面的编码函数可以看作是一个完美的哈希函数，不需要解决冲突。不过，如果事先并不知道它是完美的，也就不敢像前面一样只开一个 vis 数组。哈希技术还有很多值得探讨的地方，建议读者在网上查找相关资料。

第3种方法是用 STL 集合 t。把状态转化成 9 位十进制整数，就可以用 set<int>判重了：

```

set<int> vis;
void init_lookup_table() { vis.clear(); }
int try_to_insert(int s){
    int v = 0;
    for(int i = 0; i < 9; i++) v = v * 10 + st[s][i];
    if(vis.count(v)) return 0;
    vis.insert(v);
    return 1;
}

```

在刚才的 3 种实现中，使用 STL 集合的代码最简单，但时间效率也最低（若此时不用 -O2 优化则速度劣势更加明显）。建议读者在时间紧迫或对效率要求不太高的情况下使用，或者仅把它作为“跳板”——先写一个 STL 版的程序，确保主算法正确，然后把 set 替换成自己写的哈希表。

提示 7-16：隐式图遍历需要用 一个结点查找表来判重。一般来说，使用 STL 集合实现的代码最简单，但效率也较低。如果题目对时间要求很高，可以先把 STL 集合版的程序调试通过，然后转化为哈希表甚至完美哈希表。

某些特定的 STL 实现中还有 hash_set，它正是基于前面的哈希表，但它并不是标准 C++ 的一部分，因此不是所有情况下都可用。

例题 7-8 倒水问题 (Fill, UVa 10603)

有装满水的 6 升的杯子、空的 3 升杯子和 1 升杯子，3 个杯子中都没有刻度。在不使用其他道具的情况下，是否可以量出 4 升的水呢？

方法如图 7-15 所示。

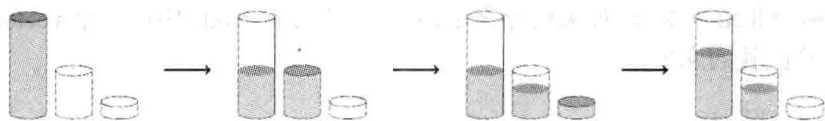


图 7-15 倒水问题：一种方法是 $(6,0,0) \rightarrow (3,3,0) \rightarrow (3,2,1) \rightarrow (4,2,0)$

注意：由于没有刻度，用杯子 x 给杯子 y 倒水时必须一直持续到把杯子 y 倒满或者把杯子 x 倒空，而不能中途停止。

你的任务是解决一般性的问题：设 3 个杯子的容量分别为 a, b, c ，最初只有第 3 个杯子装满了 c 升水，其他两个杯子为空。最少需要倒多少升水才能让某一个杯子中的水有 d 升呢？如果无法做到恰好 d 升，就让某一个杯子里的水是 d' 升，其中 $d' < d$ 并且尽量接近 d 。

($1 \leq a, b, c, d \leq 200$)。要求输出最少的倒水量和目标水量 (d 或者 d')。

【分析】

假设在某一时刻，第 1 个杯子中有 v_0 升水，第 2 个杯子中有 v_1 升水，第 3 个杯子中有 v_2 升水，称当时的系统状态为 (v_0, v_1, v_2) 。这里再次提到了“状态”这个词，它是理解很多概念和算法的关键。简单地说，它就是“对系统当前状况的描述”。例如，在国际象棋中，当前游戏者和棋盘上的局面就是刻画游戏进程的状态。

把“状态”想象成图中的结点，可以得到如图 7-16 所示的状态图 (state graph)。

由于无论如何倒，杯子中的水量都是整数（按照倒水次数归纳即可），因此第 3 个杯子的水量最多只有 $0, 1, 2, \dots, c$ 共 $c+1$ 种可能；同理，第 2 个杯子的水量一共只有 $b+1$ 种可能，第 1 个杯子一共只有 $a+1$ 种可能，因此理论上状态最多有 $(a+1)(b+1)(c+1) = 8120601$ 种可能性，有点大。幸运的是，上面的估计是不精确的。由于水的总量 x 永远不变，如果有两个状态的前两个杯子的水量都相同，则第 3 个杯子的水量也相同。换句话说，最多可能的状态数不会超过 $201^2 = 40401$ 。

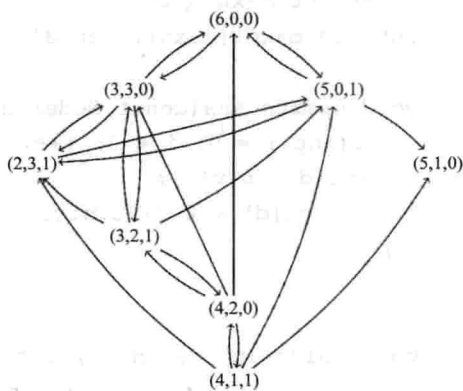


图 7-16 倒水问题的状态图

注意：本题的目标是倒的水量最少，而不是步数最少。实际上，水量最少时步数不一定最少，例如 $a=1, b=12, c=15, d=7$ ，倒水量最少的方案是 $C \rightarrow A, A \rightarrow B$ 重复 7 次，最后 C 里有 7 升水。一共 14 步，总水量也是 14。还有一种方法是 $C \rightarrow B$ ，然后 $B \rightarrow A, A \rightarrow C$ 重复 4 次，最后 C 里有 7 升水。一共只有 10 步，但总水量多达 20。

因此，需要改一下算法：不是每次取出步数最少的结点进行扩展，而是取出水量最少的结点进行扩展。这样的程序只需要把队列 `queue` 换成优先队列 `priority_queue`，其他部分的代码不变。下面的代码把状态（三元组）和 `dist` 合起来定义为了一个 `Node` 类型，是一种常见的写法。如果要打印路径，需要把访问过的所有结点放在一个 `nodes` 数组中，然后在

Node 中加一个变量 fa, 表示父结点在 nodes 数组中的下标, 而在队列中只存结点在 nodes 数组中的下标而非结点本身。如果内存充足, 也可以直接在 Node 中用一个 vector 保存路径, 省去顺着 fa 往回找的麻烦。

```
#include<cstdio>
#include<cstring>
#include<queue>
using namespace std;

struct Node {
    int v[3], dist;
    bool operator < (const Node& rhs) const {
        return dist > rhs.dist;
    }
};

const int maxn = 200 + 5;
int vis[maxn][maxn], cap[3], ans[maxn];

void update_ans(const Node& u) {
    for(int i = 0; i < 3; i++) {
        int d = u.v[i];
        if(ans[d] < 0 || u.dist < ans[d]) ans[d] = u.dist;
    }
}

void solve(int a, int b, int c, int d) {
    cap[0] = a; cap[1] = b; cap[2] = c;
    memset(vis, 0, sizeof(vis));
    memset(ans, -1, sizeof(ans));
    priority_queue<Node> q;

    Node start;
    start.dist = 0;
    start.v[0] = 0; start.v[1] = 0; start.v[2] = c;
    q.push(start);

    vis[0][0] = 1;
    while(!q.empty()) {
        Node u = q.top(); q.pop();
        update_ans(u);
        if(ans[d] >= 0) break;
        for(int i = 0; i < 3; i++)
            for(int j = 0; j < 3; j++) if(i != j) {
```



```

    if(u.v[i] == 0 || u.v[j] == cap[j]) continue;
    int amount = min(cap[j], u.v[i] + u.v[j]) - u.v[j];
    Node u2;
    memcpy(&u2, &u, sizeof(u));
    u2.dist = u.dist + amount;
    u2.v[i] -= amount;
    u2.v[j] += amount;
    if(!vis[u2.v[0]][u2.v[1]]) {
        vis[u2.v[0]][u2.v[1]] = 1;
        q.push(u2);
    }
}
}
while(d >= 0) {
    if(ans[d] >= 0) {
        printf("%d %d\n", ans[d], d);
        return;
    }
    d--;
}

int main() {
    int T, a, b, c, d;
    scanf("%d", &T);
    while(T--) {
        scanf("%d%d%d%d", &a, &b, &c, &d);
        solve(a, b, c, d);
    }
    return 0;
}

```

需要注意的是：上述算法非常直观，正确性却不是显然的。事实上，笔者目前没有找到反例，但也无法严格证明它是正确的^①。幸运的是，上述算法稍加修改，就可以得到第11章中要介绍的 Dijkstra 算法，从而保证算法的正确性。等学完 Dijkstra 算法之后，读者不妨回来再看看这道题目，相信会有新的体会。希望读者能够通过这个例题看到搜索和图论这两个看似无关的主题之间的联系。

例题 7-9 万圣节后的早晨 (The Morning after Halloween, Japan 2007, UVa1601)

$w \times h$ ($w, h \leq 16$) 网格上有 n ($n \leq 3$) 个小写字母 (代表鬼)。要求把它们分别移动到大写字母里。每步可以有多个鬼同时移动 (均为往上下左右 4 个方向之一移动)，但每

^① 如果有读者找到反例或者正确性证明，请联系笔者或者出版社，我们会在重印时更正。